

Notebook Frontend Technical Specification

Reina Al Masri

Purpose

The notebook frontend extends BioFlow-Insight to Python Jupyter notebooks (.ipynb). It reconstructs workflow graphs from notebook code in order to make hidden data dependencies explicit.

Nodes represent notebook cells or statements. Edges represent dependencies: one part of the notebook uses data produced earlier.

Usage

The frontend is called through the standard BioFlow-Insight `Workflow` API.

```
from src.workflow import Workflow

workflow = Workflow(
    workflow_file="analysis.ipynb",
    output_dir="./results",
    display_info=False
)

workflow.generate_process_dependency_graph(render_graphs=True)
```

Outputs are written under:

```
./results/graphs/
```

Main output files:

- `process_dependency_graph.json`: cell-level graph;
- `notebook_expanded_graph.json`: statement-level graph;
- `metro_map.json`: graph formatted for MetroFlow;
- optional `.dot` and `.png` files when rendering is enabled.

The graph can also be accessed directly:

```
graph = workflow.get_process_dependency_graph_dico()
```

Graph Model

The frontend produces two graph levels.

Cell-level graph. Each useful code cell becomes one node. An edge $A \rightarrow B$ means that cell B reads a variable whose latest previous definition was in cell A.

Statement-level graph. Each notebook cell becomes a box, and each top-level statement inside the cell becomes a node. This gives a more detailed view when a single cell contains several analysis steps.

Simple `if` statements are represented with condition labels on dependency edges.

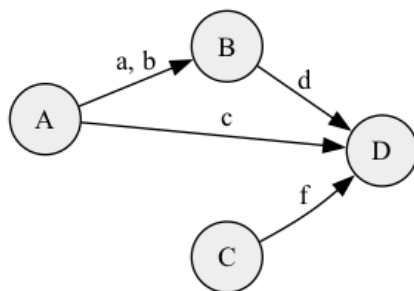


Figure 1: Cell dependency.

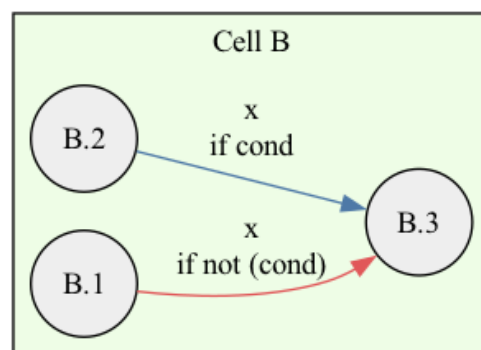


Figure 2: Conditional dependency.

Function definitions are analyzed, but the definition alone does not create a dependency edge. If a function uses an external variable, then the call cell depends on the latest previous definition of that variable.

```
[ ]: # Cell A
a = 10

[ ]: # Cell B
def f(x):
    return x + a

[ ]: # Cell C
b = f(2)
```

Figure 3: Function source.

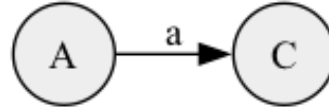


Figure 4: Generated graph.

Supported Features

The current implementation supports:

- Python Jupyter notebooks;
- ignored markdown cells;
- removal of simple magics and shell commands;
- variable definitions and variable uses;
- cell-level and statement-level dependency graphs;
- basic assignments, function definitions, and function calls;
- external variables used inside functions;
- simple top-level `if` statements;
- condition labels on supported conditional edges;
- warning messages when unsupported code is detected.

Limitations

Current limitations include:

- unknown runtime values, hidden state, and execution order;
- limited support for loops, exceptions, `match`, `with`, and comprehensions;
- limited handling of nested control flow, such as `if` statements inside loops;
- no detailed tracking of object mutations, such as `df["col"] = x` or `model.fit(X, y)`;
- technical node labels, such as `A.1` or `B.2`;
- grouping by notebook cell, not by biological or computational meaning.

Possible Extensions

Future improvements could include:

- semantic node labels based on code, comments, or markdown headings;
- more precise warnings with the exact unsupported cell;
- stronger support for loops, exceptions, and nested control flow;
- better tracking of object updates;
- function subgraphs showing parameters, internal variables, and return values;
- optional grouping by biological or computational analysis step.

Summary

The notebook frontend turns Python notebook code into workflow graphs. It makes implicit data dependencies visible through cell-level and statement-level views, while keeping outputs compatible with BioFlow-Insight and MetroFlow.

When unsupported code is found, the graph is still generated, but a warning is printed so the user knows the result may be incomplete.